

# USART Driver Document

## 1. Driver Overview

The STM32F401RE microcontroller includes multiple USART peripherals that support both synchronous and asynchronous serial communication. These peripherals provide full-duplex data exchange with configurable word length (8 or 9 bits), programmable parity (even, odd, or none), and selectable stop bits.

The USART driver for STM32F401RE supports the following features:

- Asynchronous UART and synchronous USART communication modes
- Configurable data word length (8 or 9 bits), parity (none, even, odd), and stop bits (1, 1.5, 2)
- Programmable baud rate generation supporting speeds from a few hundred baud up to 10.5 Mbit/s
- Polling-based transmission and reception for simple blocking operations
- Interrupt-driven transmission and reception with TXE, RXNE, and TC interrupt support
- DMA-supported data transfers for efficient, high-throughput communication
- Hardware flow control using CTS and RTS signals (on USART1 and USART2)
- Single-wire half-duplex mode for bidirectional communication over a single pin
- Error detection and handling for framing, noise, overrun, and parity errors

## 2. USART Driver Architecture

### **1. Peripheral Configuration Layer**

Responsible for enabling/disabling the USART peripheral clock, initializing the USART with user-specified parameters (baud rate, word length, parity, stop bits), and resetting the peripheral to its default state. Functions include `USART_PeriClockControl()`, `USART_Init()`, and `USART_DeInit()`.

## **2. Hardware Access Layer**

Directly manipulates USART registers (USART\_CR1, USART\_CR2, USART\_CR3, USART\_BRR, USART\_DR, USART\_SR) at the peripheral base address. Handles baud rate generation, transmitter/receiver enable, interrupt configuration, and status flag management.

## **3. Data Transfer Layer**

Implements blocking (polling-based) and interrupt-driven transmit/receive functions. Supports full-duplex operation, single-wire half-duplex mode, and optional DMA integration for high-throughput communication. Includes error detection for framing, noise, overrun, and parity errors.

## **4. Application API Layer**

Exposes high-level functions such as USART\_Send(), USART\_Receive(), USART\_SendReceive(), and callback mechanisms for TX complete, RX data ready, and error events. The application interacts exclusively through these APIs without direct register access, ensuring portability and ease of use.

## 3. Important USART Driver APIs

### **Initialization and Configuration**

- USART\_Init() – Initialize the USART peripheral with configuration parameters (baud rate, word length, parity, stop bits, mode).
- USART\_DeInit() – Reset and disable the USART peripheral, restoring default register values.
- USART\_PerioClockControl() – Enable or disable the clock to the USART peripheral.

## **Data Transfer**

- USART\_Send() – Transmit data in polling or interrupt mode.
- USART\_Receive() – Receive data in polling or interrupt mode.
- USART\_SendReceive() – Full-duplex transmit and receive in a single operation.

## **Control Functions**

- USART\_Enable() – Enable the USART transmitter and/or receiver.
- USART\_Disable() – Disable the USART transmitter and/or receiver.
- USART\_SetBaudRate() – Configure the baud rate generator.

## **Interrupt and Event Handling**

- USART\_IRQConfig() – Enable or disable USART-related interrupts (TXE, RXNE, TC, errors).
- USART\_IRQPriorityConfig() – Set interrupt priority for the USART peripheral.
- USART\_IRQHandler() – Interrupt service routine to handle TX, RX, and error events.

## **Callback Functions (application-level)**

- USART\_ApplicationEventCallback() – Notifies the application of events such as transmission complete, data received, or error conditions.

## 4. USART Interrupt Handling

### **Transmit-related:**

- **TXE (Transmit Data Register Empty)** – Triggered when USART\_DR is empty and ready for new data.

- **TC (Transmission Complete)** – Triggered when the last data bit has been shifted out.
- **CTS (Clear To Send)** – Triggered on CTS line state change (if hardware flow control is enabled).

#### Receive-related:

- **RXNE (Receive Data Register Not Empty)** – Triggered when received data is available in USART\_DR.
- **IDLE (Idle Line Detected)** – Triggered when the RX line remains idle after receiving data.
- **ORE (Overrun Error)** – Triggered when new data arrives before previous data is read.
- **PE (Parity Error)** – Triggered when received data fails parity check (if parity enabled).
- **FE (Framing Error)** and **NE (Noise Error)** – Triggered on frame or noise detection in multi-buffer communication.

## 5. Conclusion

The driver supports asynchronous and synchronous modes, polling and interrupt-driven transfers, hardware flow control, and comprehensive error detection. Its callback mechanism allows applications to handle communication events asynchronously, promoting non-blocking, event-driven designs. Overall, this driver simplifies USART integration, improves code portability, and reduces development time while maintaining flexibility for advanced serial communication tasks.